SPARXSTAR

Platform Architecture Amendment

PAM-001

Cross-Repository Structural Alignment · GovernanceToken TTL · Ouroboros Canonical Types

Starisian Technologies · 2026-04-14 · CONFIDENTIAL · PATENT PENDING

Supersedes all prior cross-repo DTO, contract, and TTL definitions.

Affected Repositories: Ouroboros · Sirus · Helios · Mēh₁n̥s · Dheghom · Sky

PART I

Platform Alignment and Amendment Record

1. Purpose and Scope

PAM-001 is the single authoritative record for all cross-repository structural alignment decisions in the SPARXSTAR platform. It supersedes and incorporates the earlier GovernanceToken TTL amendment and extends its scope to cover the canonical DTO field sets, signing material format, enum definitions, and contract signatures that must be identical across all six repositories before end-to-end integration can function.

This document is the specification that Copilot works against when opening PRs on any repository. When this document changes, all affected repos get a tracked migration PR. There is no other source of truth for the decisions recorded here.

2. Background

A full QC review of all six SPARXSTAR repositories conducted in April 2026 identified eight cross-repository structural mismatches that prevent end-to-end integration from functioning. The mismatches fall into two categories:

- Policy change — the GovernanceToken TTL is incompatible with 2G connectivity in the platform's primary deployment environment. A fixed 60-second TTL causes token expiry during transit on degraded connections.
- Structural drift — each repository independently defined its own provisional versions of shared types (ContextPulse, GovernanceToken, AgreementResult, ResourceSensitivity) with different field sets, backing types, casing conventions, and signing material formats. None of these provisional definitions agree with each other or with the Ouroboros canonical definitions.

Root cause
Ouroboros Sprint 0 shipped the shared type definitions, but the other five repositories were already in active development with their own provisional implementations. The provisional implementations were never reconciled against Ouroboros before cross-repo wiring was attempted. This document locks every decision required to complete that reconciliation.

3. Decision 1 — GovernanceToken TTL

The fixed 60-second GovernanceToken TTL is replaced by a connectivity-tier-aware model. This is a policy change, not a structural change. The signing material, field set, and verification logic are unchanged.

3.1 Problem

The platform's primary audience is in rural Gambia, northern Kenya, and similar low-connectivity environments. The release operation — decrypt draft, run policy packs, mint token, POST to DAL — can easily exceed 60 seconds in transit time on a degraded 2G link. The token expires before Dheghom receives it. The release guarantee is broken for the primary audience.

3.2 Decision

Replace GOVERNANCE_TOKEN_TTL_SECONDS = 60 in Ouroboros Platform.php with five tier constants. The 60-second value is retained as the minimum floor. No token may be issued with a TTL below 60 seconds.

Tier Constant	Value	Seconds	Use
GOVERNANCE_TOKEN_TTL_MIN_SECONDS	60s	60	Absolute floor — replay attack boundary
GOVERNANCE_TOKEN_TTL_HIGH_CONNECTIVITY	120s	120	Broadband / stable WiFi
GOVERNANCE_TOKEN_TTL_STANDARD	300s	300	Standard mobile / urban 3G (default)
GOVERNANCE_TOKEN_TTL_LOW_CONNECTIVITY	600s	600	2G / rural / degraded link
GOVERNANCE_TOKEN_TTL_CLI	3600s	3600	WP-CLI / SYSTEM context

ReleaseGateService in Mēhₙṇs resolves the appropriate tier from Sirus network_effective_type before calling GovernanceTokenMinter::mint(). The minter is policy-agnostic — it accepts $ttlSeconds as a caller-supplied parameter and enforces only the 60-second floor.

4. Decision 2 — Canonical ContextPulse Field Set

The canonical ContextPulse field set is defined here. All provisional ContextPulse implementations in Helios, Sirus, Dheghom, and Ouroboros are replaced by this definition when those repositories migrate to import from Ouroboros.

4.1 Canonical fields (all required)

```
final class ContextPulse
{
    public readonly string $pulse_id;     // UUID v4
    public readonly string $context_id;   // Sirus context this pulse was issued for
    public readonly string $device_id;    // Server-issued device identifier
    public readonly string $session_id;   // Active session identifier
    public readonly string $site_id;      // WordPress site ID
    public readonly string $network_id;   // Network identifier
    public readonly float  $trust_score;  // [0.0, 1.0]
    public readonly string $trust_level;  // 'NORMAL' | 'ELEVATED' | 'CRITICAL' |
'STEP_UP_REQUIRED'
    public readonly int    $issued_at;    // Unix timestamp
    public readonly int    $expires;      // Unix timestamp (absolute, not duration)
    public readonly string $sig;          // HMAC-SHA256 hex signature
}
```

4.2 Excluded fields and rationale

Field	Was in	Decision
user_id	Helios provisional	Excluded. Identity never travels in a pulse. Triple Binding is enforced via GovernanceToken.
ttl	Helios provisional	Excluded. Use expires (absolute timestamp). TTL is a duration that creates ambiguity.
behavior_flags	Ouroboros provisional	Excluded. Sirus-internal signal — not needed across the trust boundary.
geo_zone	Ouroboros provisional	Excluded. Sirus-internal — not part of the pulse contract.
session_duration	Ouroboros provisional	Excluded. Sirus-internal — not part of the pulse contract.
signature	Helios/Ouroboros provisional	Renamed to sig. Sirus canonical name wins for consistency.

4.3 Signing material

The canonical string signed by Sirus PulseGenerator and verified by Helios PulseVerifier is pipe-delimited in this exact field order:

```
pulse_id|context_id|device_id|session_id|site_id|network_id|trust_score|trust_level|issued_at|expires
```

trust_score is serialized with number_format(x, 4, '.', '') for cross-language reproducibility. Algorithm: HMAC-SHA256. Key: SIRUS_PULSE_SIGNING_KEY constant (minimum 32 bytes).

5. Decision 3 — Canonical GovernanceToken Field Set

The canonical GovernanceToken field set adds session_id and device_id to the Ouroboros provisional definition. These fields are required for Triple Binding enforcement in Dheghom.

5.1 Canonical fields

```
final class GovernanceToken
{
    public readonly string $token_id;       // UUID v4
    public readonly string $transaction_id; // UUID v4
    public readonly string $identity_id;    // Subject identity
    public readonly string $session_id;     // Triple Binding signal 1  ← ADDED
    public readonly string $device_id;      // Triple Binding signal 2  ← ADDED
    public readonly string $authority_id;   // Issuing authority
    public readonly string $ability_id;     // Permission being granted
    public readonly string $status;         // 'active' | 'revoked'
    public readonly array  $packs_enforced; // string[] — ordered pack IDs
    public readonly array  $routing_flags;  // Opaque routing metadata
    public readonly string $payload_hash;   // SHA-256 hex of governed payload
    public readonly string $signature;      // HMAC-SHA256 over canonical material
    public readonly int    $issued_at;      // Unix timestamp
    public readonly int    $expires_at;     // Unix timestamp
}
```

Note on packs_enforced type
packs_enforced is string[] (ordered list of pack IDs), not bool. The Ouroboros provisional definition used bool packs_enforced = false — this was incorrect. The Dheghom and Mēhₙ̥ns provisional definitions already use string[] — these are correct.

5.2 Canonical signing material

The canonical signing material is owned by Ouroboros as GovernanceTokenSigningMaterial::canonicalize(). The format is key=value pairs delimited by LF (\n), in this exact field order, with a version marker:

```
v=v1
token_id={token_id}
identity_id={identity_id}
session_id={session_id}
device_id={device_id}
authority_id={authority_id}
transaction_id={transaction_id}
ability_id={ability_id}
status={status}
packs_enforced={json_array}
payload_hash={payload_hash}
expires_at={expires_at}
issued_at={issued_at}
```

packs_enforced is serialized as a JSON array with JSON_UNESCAPED_UNICODE | JSON_UNESCAPED_SLASHES. Empty optional string fields are serialized as key= (empty value), not omitted. Algorithm: HMAC-SHA256.

6. Decision 4 — AgreementResult Enum Values

AgreementResult enum case values use uppercase string backing. This matches Helios PHP, the TypeScript edge worker, and the shared test vectors. Ouroboros currently uses lowercase which diverges from all three.

```
enum AgreementResult: string
{
    case ALLOW_EDGE   = 'ALLOW_EDGE';
    case ALLOW_ORIGIN = 'ALLOW_ORIGIN';
    case STEP_UP      = 'STEP_UP';
    case DENY         = 'DENY';
    case PROVISIONAL  = 'PROVISIONAL';
}
```

Wire format impact
The uppercase values are the wire format. Any system comparing AgreementResult::from('ALLOW_EDGE') must use uppercase. Helios PHP, the TypeScript worker, and shared-test-vectors.json already use uppercase. Only Ouroboros needs to change. Sirus and Dheghom do not consume AgreementResult directly — no migration needed in those repos.

7. Decision 5 — ResourceSensitivity Enum

ResourceSensitivity uses integer backing matching Helios and Sirus. Ouroboros currently uses string backing which diverges from both.

```
enum ResourceSensitivity: int
{
    case LEVEL_1 = 1;  // Public / General
    case LEVEL_2 = 2;  // Creator / Financial
    case LEVEL_3 = 3;  // Heritage / Sovereign
}
```

8. Decision 6 — HeliosClientInterface Method Signatures

The HeliosClientInterface is updated to match what Helios AgreementEvaluator actually implements. The current Ouroboros definition has a $proof parameter and is missing $now and $zone, making it impossible for Helios to satisfy the interface.

```
interface HeliosClientInterface
{
    public function evaluate(
        ?ContextPulse        $pulse,
        ResourceSensitivity  $sensitivity,
        int                  $now,
        string               $zone         // 'edge' | 'origin'
    ): AgreementResult;

    public function verifyDevice(string $deviceId): bool;
    public function getTrustState(string $deviceId): string;
    public function validateSession(string $sessionId, string $deviceId): bool;
}
```

The $proof parameter is removed. The Helios AgreementEvaluator does not accept a proof artefact in its evaluate() method — proof handling is internal to the step-up flow. The $now and $zone parameters are added to match the actual evaluator signature.

9. Decision 7 — Loader Sky Class Expectation

The Ouroboros loader currently expects Starisian\\Sparxstar\\Sky\\Engine which does not exist in the Sky repository. Sky is a WordPress plugin with a procedural entry point, not a namespaced class. The loader is updated to check for the SpxAPI class which is Sky's primary REST controller and is guaranteed to exist after Sky loads.

10. Implementation Checklist

All items are tracked against Ouroboros as the primary change repository. Component-level migrations follow after Ouroboros changes are merged.

#	Change	Repository	Pass Condition
1	Remove GOVERNANCE_TOKEN_TTL_SECONDS = 60	Ouroboros	Constant absent; five tier constants present
2	Add five TTL tier constants to Platform.php (Decision 1)	Ouroboros	All five constants present and correct

#	Change	Repository	Pass Condition
3	Update ContextPulse DTO to canonical field set (Decision 2)	Ouroboros	Fields match §4.1 exactly
4	Update GovernanceToken DTO — add session_id, device_id; fix packs_enforced type (Decision 3)	Ouroboros	Fields match §5.1; packs_enforced is string[]
5	Add GovernanceTokenSigningMaterial::canonicalize() (Decision 3)	Ouroboros	Canonical format matches §5.2
6	Update AgreementResult enum values to uppercase (Decision 4)	Ouroboros	ALLOW_EDGE not allow_edge; all five cases uppercase
7	Update ResourceSensitivity to int backing (Decision 5)	Ouroboros	LEVEL_1=1, LEVEL_2=2, LEVEL_3=3
8	Update HeliosClientInterface::evaluate() signature (Decision 6)	Ouroboros	Signature matches §8
9	Update loader loadSky() class check (Decision 7)	Ouroboros	Checks SpxAPI not Sky\Engine
10	Update ai_manifest.json to reflect new field counts	Ouroboros	Symbol count correct
11	Align GovernanceTokenMinter signing material to GovernanceTokenSigningMaterial	Mēhiṇs	Minted tokens pass Gate 1 in Dheghom
12	Implement TTL floor guard and ReleaseGateService tier resolution (Decision 1)	Mēhiṇs	Floor test passes; 2G resolves 600s — COMPLETE
13	Migrate Helios provisional ContextPulse to Ouroboros import	Helios	Local ContextPulse class removed
14	Migrate Helios AgreementResult to Ouroboros import; update string values	Helios	All match tests pass with uppercase values
15	Implement Helios AgreementEvaluator against updated HeliosClientInterface	Helios	Shared test vectors 10/10 pass
16	Migrate Sirus provisional ContextPulse to Ouroboros import	Sirus	Local ContextPulse class removed; PulseGenerator updated
17	Migrate Dheghom provisional GovernanceToken to Ouroboros import	Dheghom	Local GovernanceToken class removed

#	Change	Repository	Pass Condition
18	Update TokenValidator to use GovernanceTokenSigningMaterial::canonicalize()	Dheghom	Already done in PR #6 — verify against Ouroboros class
19	Sky: no Ouroboros DTO migration required	Sky	No action — Sky does not consume GovernanceToken or ContextPulse
20	Update shared-test-vectors.json with uppercase AgreementResult values	Ouroboros	Jest 10/10 pass; PHP shared vector test passes

11. Migration Order

Phase	Scope	Prerequisite
Phase 1	Items 1–10 — all Ouroboros changes	None. Ouroboros depends on nothing.
Phase 2	Items 11–12 — Mēhiṇs signing material and TTL	Phase 1 complete. Ouroboros GovernanceTokenSigningMaterial must exist.
Phase 3	Items 13–15 — Helios migration	Phase 1 complete. Ouroboros ContextPulse and AgreementResult must be canonical.
Phase 4	Item 16 — Sirus migration	Phase 1 and Phase 3 complete. Helios PulseVerifier must accept canonical field set.
Phase 5	Items 17–18 — Dheghom migration	Phase 1 and Phase 2 complete. Ouroboros GovernanceToken must carry session_id and device_id.
Phase 6	End-to-end integration test	All phases complete.

12. Security Considerations

Property	Impact	Notes
Replay attack prevention	Preserved	60-second TTL floor retained. No token is reusable after expiry regardless of tier.
ContextPulse integrity	Preserved	Signing material field set and HMAC-SHA256 algorithm unchanged.
GovernanceToken integrity	Strengthened	session_id and device_id added to signing material via canonicalize(). Token is more tightly bound.
Triple Binding	Enabled	session_id and device_id now travel in the token, enabling Dheghom Gate enforcement end-to-end.

Property	Impact	Notes
Identity exclusion from pulse	Preserved	identity_id is not present in the canonical ContextPulse field set.
Fail-closed loader	Preserved	Loader logic unchanged. exit(1) on any component failure remains.

PART II

GovernanceToken TTL — Connectivity-Tier-Aware Model

Original Platform Architecture Amendment · April 14, 2026

This section preserves the original TTL amendment. The decisions in Part I supersede and extend this content. It is retained for audit and provenance.

1. Purpose and Scope

This amendment is a platform-level change. It supersedes the fixed 60-second GovernanceToken TTL defined in the Mēh₁n̥s DVE Core Spec v3.0 and referenced across Ouroboros Integrity, Sirus, Helios, and Dheghom. It replaces that fixed value with a connectivity-tier-aware model that preserves all security guarantees of the original design while remaining functional for the platform's primary deployment environment: 2G and low-connectivity networks in rural Africa.

2. Problem Statement

The 60-second TTL was specified as a replay attack prevention window. The implicit assumption is that a token is issued, travels to the DAL, and is consumed within a single fast request cycle. On a reliable broadband connection this is correct.

The platform's primary audience is in rural Gambia, northern Kenya, and similar low-connectivity environments. The Ouroboros Integrity Spec v1.0 explicitly identifies Cloudflare nodes in Lagos, Nairobi, and Johannesburg as a basic usability requirement for 2G connectivity. The release operation — decrypt draft, run policy packs through Mēh₁n̥s, mint GovernanceToken, POST token-bearing payload to Dheghom — can easily exceed 60 seconds in transit time alone on a degraded 2G link.

Failure mode on 2G

- Mēh₁n̥s mints a GovernanceToken with a 60-second TTL.
- The token-bearing payload travels from origin to Dheghom over a degraded 2G link.
- Transit time exceeds 60 seconds.
- Dheghom TokenValidator Gate 3 (expiry check) throws TokenValidationException.
- The release fails. The user receives an error.
- On a 2G connection, the retry is likely to fail for the same reason.
- The record remains locked in draft state. The release guarantee is broken for the primary audience.

This is not a marginal edge case. It is a structural failure for the communities the platform exists to serve.

3. Design Principle

The Sirus Context Engine Spec v3.0 §11 already defines a connectivity-tier-aware TTL model for ContextPulse with documented working defaults by tier. This amendment applies the same pattern to GovernanceToken TTL. The design principle is identical: the TTL is a caller-supplied input resolved from Sirus connectivity context. The component that mints the token remains policy-agnostic.

Two invariants from the original design are preserved without modification:

- The 60-second value is retained as the minimum floor. No token may be issued with a TTL below 60 seconds. This preserves the replay attack prevention boundary unconditionally.
- The GovernanceTokenMinter remains policy-agnostic. It does not resolve connectivity tier. TTL is a required caller input. The ReleaseGateService owns TTL resolution from Sirus context before calling mint().

4. Connectivity-Tier TTL Values

Tier Constant	Value	Seconds	Use
GOVERNANCE_TOKEN_TTL_MIN_SECONDS	60s	60	Absolute floor — replay attack boundary
GOVERNANCE_TOKEN_TTL_HIGH_CONNECTIVITY	120s	120	Broadband / stable WiFi
GOVERNANCE_TOKEN_TTL_STANDARD	300s	300	Standard mobile / urban 3G (default)
GOVERNANCE_TOKEN_TTL_LOW_CONNECTIVITY	600s	600	2G / rural / degraded link
GOVERNANCE_TOKEN_TTL_CLI	3600s	3600	WP-CLI / SYSTEM context

Hard floor — never negotiable
Minimum TTL: 60 seconds. The GovernanceTokenMinter MUST throw InvalidArgumentException if $ttlSeconds < 60. This floor exists to prevent replay attacks. No connectivity tier, no deployment mode, and no administrative override may reduce it.

5. ReleaseGateService — TTL Resolution (Mēhₜṇs)

ReleaseGateService is the sole caller of GovernanceTokenMinter::mint() in the release path. The following resolution logic is implemented in ReleaseGateService::resolveTtlSeconds(ContextPulse $pulse):

```
return match ($pulse->network_effective_type) {
    '4g', 'wifi'    => Platform::GOVERNANCE_TOKEN_TTL_HIGH_CONNECTIVITY,
    '3g'            => Platform::GOVERNANCE_TOKEN_TTL_STANDARD,
    '2g', 'slow-2g' => Platform::GOVERNANCE_TOKEN_TTL_LOW_CONNECTIVITY,
    'cli', 'system' => Platform::GOVERNANCE_TOKEN_TTL_CLI,
    default         => Platform::GOVERNANCE_TOKEN_TTL_STANDARD,
};
```

PART III

Implementation Scorecard

Last updated: April 22, 2026 · Phase 1 complete — all four PRs merged

Legend	Meaning
DONE	Merged, passing, complete
OPEN	Not yet built
BLOCKED	Cannot start until prerequisite phases complete

Ouroboros — sparxstar-ouroboros-integrity

Phase 1 (items 1–10) — STATUS: COMPLETE

Change	Status	Notes
1 Remove GOVERNANCE_TOKEN_TTL_SECONDS = 60	DONE	Constant absent; five tier constants present
2 Add five TTL tier constants to Platform.php	DONE	All five present and correct
3 Update ContextPulse DTO to canonical field set	DONE	Fields match PAM-001 §4.1 exactly
4 Update GovernanceToken DTO — fix packs_enforced type; session_id/device_id in routingFlags	DONE	string[]; session_id/device_id NOT on DTO (architectural decision April 21 2026)
5 Add GovernanceTokenSigningMaterial::build()	DONE	Canonical format matches PAM-001 §5.2
6 Update AgreementResult enum to uppercase	DONE	ALLOW_EDGE not allow_edge; all five cases
7 Update ResourceSensitivity to int backing	DONE	LEVEL_1=1, LEVEL_2=2, LEVEL_3=3
8 Update HeliosClientInterface::evaluate() signature	DONE	Signature matches PAM-001 §8
9 Update loader loadSky() class check	DONE	Checks SpxAPI not Sky\Engine
10 Update ai_manifest.json symbol counts	DONE	Symbol count correct

Mēhᵢᵢnˌs — sparxstar-mehns-dve-core

Phase 2 (items 11–12) — STATUS: COMPLETE

Change	Status	Notes
11 Align GovernanceTokenMinter signing material to GovernanceTokenSigningMaterial from Ouroboros	DONE	PR #5 merged. Local GovernanceTokenSigningMaterial deleted. Ouroboros build() imported and used.
12 Implement TTL floor guard and ReleaseGateService tier resolution (PAM-001 Decision 1)	DONE	PR #6 merged. All 7 tiers + floor test passing. No hardcoded 60s remains.

Pass conditions met:

- ReleaseGateService::resolveTtlSeconds(ContextPulse) implemented
- wifi/4g → GOVERNANCE_TOKEN_TTL_HIGH_CONNECTIVITY (120)
- 3g → GOVERNANCE_TOKEN_TTL_STANDARD (300)
- 2g/slow-2g → GOVERNANCE_TOKEN_TTL_LOW_CONNECTIVITY (600)
- cli/system → GOVERNANCE_TOKEN_TTL_CLI (3600)
- unknown/null → GOVERNANCE_TOKEN_TTL_STANDARD (300)
- Minter floor test passes (TTL < 60 throws InvalidArgumentException)

Helios — sparxstar-helios-trust

Phase 3 (items 13–15) — STATUS: OPEN (unblocked by Phase 1 merge)

Change	Status	Notes
13 Migrate Helios provisional ContextPulse to Ouroboros import	OPEN	Delete local ContextPulse.php. Import Ouroboros\DTOs\ContextPulse. PulseVerifier updated to canonical type.
14 Migrate Helios AgreementResult to Ouroboros import; update string values to uppercase	OPEN	Delete local AgreementResult.php. Import Ouroboros\Enums\AgreementResult. ALLOW_EDGE, ALLOW_ORIGIN, STEP_UP, DENY, PROVISIONAL.
15 Implement Helios AgreementEvaluator against updated HeliosClientInterface	OPEN	HeliosClientInterface now uses $now and $zone parameters; $proof removed. AgreementEvaluator must satisfy updated interface from Ouroboros.

Sirus — sparxstar-sirus-context

Phase 4 (item 16) — STATUS: BLOCKED on Phase 3

Change	Status	Notes
16 Migrate Sirus provisional ContextPulse to Ouroboros import	OPEN	Delete local ContextPulse class. PulseGenerator updated to import from Ouroboros.

Change	Status	Notes
		network_effective_type must be present — ReleaseGateService reads it for TTL tier resolution.

Dheghom — sparxstar-dheghom-dve-core

Phase 5 (items 17–18) — STATUS: BLOCKED on Phase 2

Change	Status	Notes
17 Migrate Dheghom provisional GovernanceToken to Ouroboros import	OPEN	Delete local GovernanceToken class. Import Ouroboros\DTOs\GovernanceToken. TokenValidator uses canonical type. NOTE: session_id/device_id NOT on DTO — they are in routingFlags. PR #6 enforces this via session store.
18 Verify TokenValidator uses GovernanceTokenSigningMaterial::build() from Ouroboros	OPEN	PR #6 merged. Verify TokenValidator imports and uses Ouroboros build(). If local copy was deleted in PR #6, this may already be done.

Sky — sparxstar-sky-dve-core

Phase N/A — STATUS: NO CHANGES REQUIRED

Sky does not consume GovernanceToken or ContextPulse directly. No PAM-001 migration work required for Sky in Phases 1–5.

Phase 6 — End-to-End Integration Test

STATUS: BLOCKED (all phases must complete)

Pass condition:

- Sirus mints a ContextPulse with correct canonical field set
- Helios verifies the pulse using Ouroboros PulseVerifier
- Mēhₒṇs mints a GovernanceToken with tier-aware TTL via ReleaseGateService
- Dheghom validates the token through all three gates
- Token minted by Mēhₒṇs passes Dheghom TokenValidator Gates 1, 2, and 3

Current Build Order

IMMEDIATE — both unblocked by Phase 1 and Phase 2 completing:

- Helios: Open Phase 3 PR — migrate ContextPulse and AgreementResult to Ouroboros imports (items 13–15)

AFTER HELIOS PHASE 3 COMPLETE:

- Sirus: Open Phase 4 PR — migrate ContextPulse to Ouroboros import (item 16)

AFTER MEHNS PHASE 2 COMPLETE (already done):

- Dheghom: Open Phase 5 PR — migrate GovernanceToken to Ouroboros import (items 17–18)

AFTER ALL PHASES COMPLETE:

- Phase 6: End-to-end integration test

PART IV

OPEN ARCHITECTURAL DECISION

Two Distinct Token Types Required

Release Receipt vs Artifact Governance Declaration

Caught during PR review of Dheghom PR #6, April 2026. Decision recorded to prevent the two token types from being further conflated in the codebase before the distinction is architecturally resolved.

STATUS: OPEN — Requires spec session before implementation.

Decision Needed

The current platform has one GovernanceToken. It needs two distinct instruments. These are not variants of the same object. They are different legal and technical constructs with different lifecycles, different authorities, and different purposes.

Token Type 1 — Release Receipt (currently built)

Also called: GovernanceToken (current implementation in Mēh₁n̥s/Dheghom)

Purpose	Proof that a specific payload cleared Mēh₁n̥s policy evaluation at a specific moment in time and is cleared for vault ingestion.
Lifecycle	Short-lived. Minutes to hours depending on connectivity tier (PAM-001). Minimum 60 seconds. Maximum 3600 seconds (CLI/SYSTEM context).
Minted by	GovernanceTokenMinter (Mēh₁n̥s)
Consumed by	Dheghom TokenValidator at write time
Expires	Yes. Deliberately. Replay attack prevention.
What it answers	"Did this exact payload clear governance sieve at this moment under this authority?"

This is already built across Mēh₁n̥s and Dheghom PRs.

Token Type 2 — Artifact Governance Declaration (NOT YET BUILT)

Working name: ArtifactGovernanceDeclaration / SovereigntyDeclaration

Purpose	The persistent, artifact-bound declaration of who owns this artifact, under what law it exists, and what rules govern its use — from the moment of release until the authorized authority explicitly changes them.

Lifecycle	Permanent. Does not expire. "Until I say otherwise." Survives the artifact. Travels with it across systems. Amended only by explicit, cryptographically-verified authority action.

Minted by	TBD — likely Mēh₁n̥s at release time, alongside the Release Receipt, but stored permanently in Dheghom rather than consumed and discarded.

What it answers	"What are the permanent rules governing this artifact, under whose authority, and what law applies?"

Preliminary field set

Field	Purpose
artifact_id	The artifact this declaration governs
authority_id	The sovereign authority under whose jurisdiction this falls (individual, community, tribal nation, etc.)
identity_id	The contributor who made the declaration
jurisdiction	The legal/cultural framework that governs this artifact (e.g. Cahuilla tribal law, CARE principles, TK Labels, Creative Commons variant, proprietary)
tk_labels	Traditional Knowledge Label assertions (if applicable)
care_principles	CARE data governance principles (if applicable)
royalty_terms	How commercial use is compensated (if applicable)
geographic_scope	Where the artifact may be used / distributed
permitted_uses	What the artifact may be used for
prohibited_uses	What is explicitly forbidden
ai_training_rights	Whether the artifact may be used for AI training, and under what conditions
voice_reconstruction_rights	Whether the contributor's token-level data may be used for voice reconstruction
amendment_authority	Who may change these terms and how
declared_at	Unix timestamp of original declaration

Field	Purpose
signature	HMAC or cryptographic proof of declaration authority

Why This Matters

The current architecture conflates two things that must be distinct:

- "Did this payload clear the sieve right now?" — Release Receipt (short-lived)
- "What are the permanent rules for this artifact?" — Declaration (permanent)

A song recorded by a Cahuilla elder cannot be governed by a 300-second token. The tribal sovereignty rules that apply to that song do not expire with the HMAC. They apply forever, or until the tribal authority explicitly amends them.

The Release Receipt is the operational key that opens the vault door. The ArtifactGovernanceDeclaration is the deed that says who owns what's inside.

Relationship to Locket

The Locket holds the contributor's personal sovereignty declarations before they are attached to a specific artifact. The ArtifactGovernanceDeclaration is the artifact-level instantiation of those declarations at release time. The Locket is the source of truth. The Declaration is the bound instrument.

Relationship to Release Receipt

Both are minted at release. The Release Receipt expires. The Declaration does not. They are issued together but are separate objects with separate lifecycles.

Amendment Record

Document ID	PAM-001
Type	Platform Alignment and Amendment Record
Status	Phase 1 Complete · Phase 2 Complete · Phase 3 Open
Date	2026-04-14 (original) · 2026-04-22 (scorecard updated)
Author	Starisian Technologies
Supersedes	Previous GovernanceToken TTL amendment (TTL section); all provisional cross-repo DTO definitions
Primary implementation repo	sparxstar-ouroboros-integrity (all structural changes)

Patent family	Patent Family C — Multi-Tiered Executable Governance; Patent Family A — Brain-Sieve Architecture